

Claude Code Subagents

A separate Claude per job — own context, own instructions, own tools.
Coordinator patterns, parallel fan-out and every setting that matters —
reference guide

WHAT'S INSIDE

1. Why subagents — the context-clutter problem
2. What a subagent is — and what it starts with
3. Built-in subagents
4. Anatomy of an agent file & every frontmatter field
5. Where agents live — scope & priority
6. Controlling capabilities — tools, permissions, MCP, skills, memory, hooks
7. Choosing a model
8. Invoking & delegating — automatic, @-mention, --agent
9. Running them — parallel, background, nesting, forks, resume
10. Worked example A — coordinator + tc-subagent
11. Worked example B — spec-reviewer + security-agent
12. Subagents vs Skills vs Commands vs CLAUDE.md
13. Gotchas, glossary & self-test

Sources: Anthropic's Claude Code documentation ("Create custom subagents"), the Anthropic Academy subagents course, cross-checked against the Episode script and the project's own agent files. Version notes reflect the docs at the time of writing — check the docs for newer behaviour.

Why Subagents — the Context-Clutter Problem

Everything Claude reads while investigating stays in your conversation — and is resent on every turn

A language model is stateless: on every new turn the whole conversation so far — your messages, the system prompt, every file read, every tool result — is sent to it again as input. So when you ask “Review my framework for security issues, performance issues and coverage gaps”, Claude has to open dozens of page objects, fixtures, specs and config files. Two things then pile up in the main context window:

- **The raw content** of every file it read.
- **The intermediate work** — searches, tool results, analysis and reasoning while hunting for the issues.

Almost all of that was only needed during the investigation. But it stays, and it is carried into every later turn — even an unrelated one such as “add mobile Safari to [playwright.config.ts](#)”.

Four things go wrong when the window fills with leftovers

PROBLEM	WHAT HAPPENS	WHY IT HURTS
1 • Distraction	Security-review text (tokens, auth logic, credential checks) is still in context when you ask a config question.	The model can be statistically pulled toward irrelevant assumptions and mix contexts.
2 • Degradation	The more that is held in the window, the less sharply the model focuses on any one detail — especially material in the middle of a long conversation.	Quality drops well before the window is actually full. This is a property of LLMs generally, not just Claude.
3 • Lossy auto-compact	Near the limit, Claude Code summarises older turns and drops the rest to make room.	Fine details of the 50-file review may not survive, or may be misremembered later.
4 • Sequential + shared context	Many similar jobs run one after another in one window (e.g. 3 stories → 3× the time).	Job 3 drifts toward the phrasing of jobs 1 and 2, and wall-clock time scales linearly.

WARNING · A BIGGER WINDOW DOES NOT MAKE THIS GO AWAY

Even with a 1M-token window, 40–50 files will not “fill it up” — and large windows are not available on every model or plan. The stronger argument is **clutter, not overflow**: leftover material still dilutes focus, still costs tokens on every turn, and long sessions still auto-compact eventually. Say “a large chunk of the codebase” rather than implying overflow.

The fix: hand the job to someone else

Anthropic positions subagents for exactly this: a side task that would flood the main conversation with search results, logs or file contents you will not need again is done in a **separate context window**, and only the summary comes back. Think of the main session as a project lead and the subagent as a contractor brought in for one narrow job.

BENEFIT (PER ANTHROPIC)	WHAT IT MEANS FOR YOU
Preserve context	Exploration and implementation noise stays out of the main conversation.
Enforce constraints	Limit which tools a given worker can use (e.g. read-only reviewer).
Reuse configurations	User-level agents in <code>~/.claude/agents/</code> work across every project.
Specialise behaviour	A focused system prompt per domain (security review, debugging, data analysis).
Control costs	Route routine work to faster, cheaper models such as Haiku.

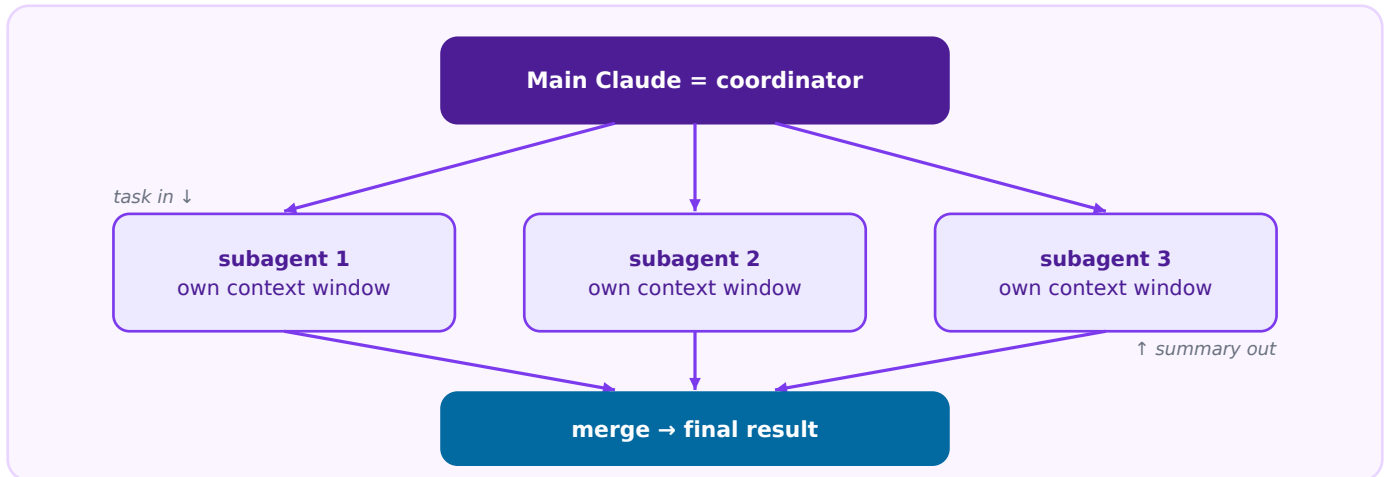
KEY IDEA · SKILL VS SUBAGENT, IN ONE LINE

A **skill is text** — a chapter loaded into the context you are already in. A **subagent is a whole new context** — a colleague you hand a task to.

What a Subagent Actually Is

A separate Claude with its own context window, system prompt, tool access and permissions

A subagent is defined as a Markdown file with YAML frontmatter. When Claude meets a task that matches its description, it **delegates**: the subagent works independently in its own window and returns a result. Its requests still count toward the same usage limits as your main conversation.



What a (non-fork) subagent starts with

ITEM	DETAIL
System prompt	The agent's own Markdown body plus basic environment details (e.g. working directory) — not the Claude Code system prompt.
Task message	The delegation prompt Claude writes when it hands off the work — its main input.
CLAUDE.md files	The full CLAUDE.md hierarchy the main session loads (user, project, local, managed, AGENTS.md). Built-in Explore and Plan skip it; a custom agent can opt out with <code>omitClaudeMd: true</code> .
Git status	A snapshot taken when it starts. Explore and Plan skip it.
Preloaded skills	Full content of any skill named in the <code>skills</code> field.
Sibling roster	Only if it has the <code>SendMessage</code> tool and another named agent exists.

What it does NOT get

- Your **conversation history** — this is the isolation you are buying.
- Skills you already invoked, and files Claude already read.
- Your **output style**, and the main conversation's **auto memory** (use the `memory` field for its own).
- The parent's window size — its context window is sized by **its own model**.

CORRECTION · “BLANK CONTEXT” IS NOT QUITE EMPTY

The script says a subagent “sees only what you hand it”. Precisely: it sees no **conversation history**, but **CLAUDE.md and git status still load** by default.

In the coordinator design that matters: `tc-subagent` will also read the coordinator plan in `CLAUDE.md` (“I do NOT generate test cases myself”). If that ever confuses a worker, add `omitClaudeMd: true` to its frontmatter (Claude Code v2.1.271+) and put everything it needs in its own file and prompt.

When Claude spawns a subagent it does so with a tool call. The tool is named **Agent** (renamed from **Task** in v2.1.63; old `Task(...)` references still work as aliases). Fan-out is simply several Agent calls in the same turn.

Built-in Subagents

Claude Code ships helpers it uses automatically — you rarely call them yourself

AGENT	MODEL	TOOLS	USED FOR
Explore	Inherits the main conversation's model, capped at Opus on the Claude API (from v2.1.198; previously Haiku).	Read-only — Write and Edit denied.	File discovery, code search, codebase exploration. Claude sets a thoroughness level: quick, medium or very thorough.
Plan	Inherits the main model.	Read-only.	Research during plan mode so exploration output stays out of the read-only main thread.
general-purpose	Env-var model if set, otherwise the main model.	Every tool available to subagents.	Complex tasks needing both exploration and action, or several dependent steps.
claude	None of its own; follows the normal model order.	Every tool available to subagents.	Catch-all when nothing more specific fits; default agent for dispatched background sessions.
statusline-setup	Sonnet	—	Runs when you use <code>/statusline</code> .
claude-code-guide	Haiku	—	Answers questions about Claude Code features.

- A user or project agent named **Explore** **overrides** the built-in one and keeps its own `model` — define one with `model: haiku` to keep exploration cheap.
- Block one type: `permissions.deny` with `Agent(Explore)`. Block delegation entirely: deny the `Agent` tool.
- Remove only Explore and Plan: `CLAUDE_CODE_DISABLE_EXPLORE_PLAN_AGENTS=1` (v2.1.198+).
- Explore and Plan are one-shot: they return no agent ID, so they cannot be resumed.

Anatomy of an Agent File

YAML frontmatter that configures the worker, then a Markdown body that becomes its system prompt

`.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Reviews code for quality and best practices. Use after writing or modifying code.
tools: Read, Glob, Grep
model: sonnet
---
```

You are a code reviewer. When invoked, analyze the code and provide specific, actionable feedback on quality, security, and best practices.

Only name and description are required. The body is the system prompt — the worker gets that plus environment details, not the Claude Code prompt.

Every frontmatter field

FIELD	REQ?	WHAT IT DOES
<code>name</code>	Yes	Unique identifier (hooks receive it as <code>agent_type</code>). Filename need not match. Cannot contain <code>:</code> (reserved for plugin scoping) or start with <code>-</code> .
<code>description</code>	Yes	When Claude should delegate to this agent. The only part always in the main context — the routing lever.
<code>tools</code>	No	Allowlist, comma-separated or YAML list. Omitted = inherits every tool available to subagents. Supports <code>Agent(type)</code> and MCP patterns like <code>mcp_github</code> .
<code>disallowedTools</code>	No	Denylist removed from the inherited or listed set. A specifier such as <code>Bash(git push *)</code> still removes the whole tool.
<code>model</code>	No	<code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , a full model ID, or <code>inherit</code> . Omitted → the model order in Part 7.
<code>permissionMode</code>	No	<code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , <code>plan</code> (<code>manual</code> = alias for <code>default</code>). Ignored for plugin agents.
<code>maxTurns</code>	No	Stops after N agentic turns; output is marked partial and Claude can resume it.
<code>skills</code>	No	Skills whose full content is injected at startup.
<code>mcpServers</code>	No	Servers scoped to this agent: a name (reuses the session connection) or an inline definition. Ignored for plugin agents.
<code>hooks</code>	No	Lifecycle hooks active only while this agent runs. Ignored for plugin agents.
<code>memory</code>	No	Persistent memory scope: <code>user</code> , <code>project</code> or <code>local</code> .
<code>background</code>	No	<code>true</code> keeps it in the background even if Claude asks for the foreground.
<code>omitClaudeMd</code>	No	<code>true</code> launches it without user/project/local CLAUDE.md (v2.1.271+).
<code>effort</code>	No	<code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code> (model-dependent); overrides session effort.
<code>isolation</code>	No	<code>worktree</code> runs it in a temporary git worktree; cleaned up if nothing changed.
<code>color</code>	No	Display colour in the task list: red, blue, green, yellow, purple, orange, pink or cyan.
<code>initialPrompt</code>	No	Auto-submitted first turn when the agent runs as the main session (<code>--agent</code>).

WARNING · SILENT FAILURES TO KNOW ABOUT

Field names are exact camelCase (`maxTurns`, `disallowedTools`). An unrecognised field is **ignored without an error**.

A file is silently skipped if: it has **no** `name` (treated as documentation), the opening `---` is not line 1, `name` starts with `-` or contains `:`, it has a `name` but **no** `description`, or the YAML does not parse.

Diagnose with `claude --debug`; pre-check a folder with `claude plugin validate .claude/agents` (v2.1.233+).

Creating one

- **Ask Claude** to write it (say the scope and what tools/model you want) or write the file by hand.
- As of v2.1.198 the `/agents` wizard is gone; running it just points you to the files. On v2.1.197 and earlier it still opens the wizard.
- Claude Code watches `~/.claude/agents/` and `.claude/agents/` and picks up edits within seconds. **Restart** only for: the first agent in a brand-new `agents` directory, agents inside `--add-dir` folders, or sessions started with `--disable-slash-commands`.

A body convention worth copying

The four headings in the Episode script are **your convention, not a Claude Code requirement** — but a good one: **My role** (one narrow job) · **My inputs** (the exact payload shape) · **Execution steps** · **Hard constraints** (guardrails such as “valid JSON array only, no prose”, “do not write files”).

PART 5 · LOCATION & PRIORITY

Where Agents Live

The folder decides who can use an agent; the highest-priority location wins on a name clash

LOCATION	SCOPE	PRIORITY	HOW TO CREATE
Managed settings	Organisation-wide	1 (highest)	Deployed by admins via managed settings
<code>--agents</code> CLI flag	Current session only	2	JSON passed at launch; not saved to disk
<code>.claude/agents/</code>	Current project	3	Ask Claude, or create the file. Commit it so the team shares it
<code>~/.claude/agents/</code>	All your projects	4	Ask Claude, or create the file
Plugin <code>agents/</code> folder	Where plugin is enabled	5 (lowest)	Installed with plugins

- **Project agents** are found by walking up from the working directory to the repo root; if the same `name` appears more than once, the one **closest** to the working directory wins.
- Folders are scanned **recursively** — `agents/review/` and `agents/research/` are fine. Identity comes only from `name`, so keep names unique across the tree.
- **Plugin agents** get a scoped identifier (`my-plugin:review:security`) and, for security, ignore `hooks`, `mcpServers` and `permissionMode`. Copy the file into `.claude/agents/` if you need them.
- `--add-dir` folders also contribute their `.claude/agents/`.

CLI-defined agent (session only) — ``prompt`` is the system prompt

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after code changes.",
    "prompt": "You are a senior code reviewer. Focus on quality, security, best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  }
}'
```

Controlling Capabilities

Tools, permissions, MCP servers, skills, memory and hooks — per agent

Tools: allowlist, denylist, and what is always removed

```
tools: Read, Grep, Glob, Bash          # allowlist - nothing else (no Edit, Write or MCP)
disallowedTools: Write, Edit           # denylist - everything inherited EXCEPT these
disallowedTools: mcp__github           # drop a whole MCP server, keep the rest
```

- If both are set, `disallowedTools` is applied **first**, then `tools` is resolved against what remains. A tool in both is removed.
- **Always removed from subagents**, even if listed: `AskUserQuestion`, `EnterPlanMode`, `ExitPlanMode` (unless `permissionMode: plan`), `ScheduleWakeup`, `WaitForMcpServers`, `Workflow`, `EndConversation`, and `Agent` once the depth limit is reached.
- **Background** subagents also get a smaller built-in tool set (Read, Grep, Glob, LSP, Bash, Edit, Write, WebFetch, WebSearch, Skill, and a few more) — same definition, different tools in foreground vs background.
- To keep Bash but block specific commands, use a `permissions.deny` rule such as `Bash(git push *)` — it applies to the main session and to subagents.

KEY IDEA · TOOLS: REALLY RESTRICTS — UNLIKE A SKILL'S ALLOWED-TOOLS

In a **skill**, `allowed-tools: Read` only pre-approves Read (it does not lock anything out). In a **subagent**, `tools: Read` is an allowlist: the worker genuinely cannot edit files or run commands. That is why `spec-reviewer (tools: Read)` is safe to point at your specs.

No frontmatter setting means “zero tools”. Omit `tools` and it inherits everything; list only names that do not resolve and, from v2.1.208, Claude Code usually refuses to launch it. For a worker that should call nothing, the enforcement is the prose in its Hard constraints (“do not call tools”) — or, better, give it only the tools it needs.

Limiting which agents a coordinator can spawn

`tools: Agent(worker, researcher), Read, Bash` is an **allowlist of spawnable types** — but it only applies to an agent running as the **main thread** via `claude --agent`. Inside a subagent definition, listing `Agent` lets it spawn its own subagents (while the depth limit allows) and any type list in the brackets is ignored. Omit `Agent` entirely and it cannot spawn anything.

Permission modes

MODE	BEHAVIOUR
<code>default</code> (alias <code>manual</code>)	Prompts for permission.
<code>acceptEdits</code>	Auto-accepts file edits and common filesystem commands in the working directory / added directories.
<code>auto</code>	A background classifier reviews commands and protected-directory writes.
<code>dontAsk</code>	Auto-denies permission prompts; explicitly allowed tools still work.
<code>bypassPermissions</code>	Skips prompts — a subagent runs in it only when the main conversation does .
<code>plan</code>	Read-only exploration.

Precedence: if the main session is in `bypassPermissions`, `acceptEdits` or `auto`, the subagent runs in that same mode and its own `permissionMode` is **ignored**. If the main session is in `default`, `dontAsk` or `plan`, the agent's setting applies (except `bypassPermissions`). Unset = inherit.

Preload skills into a subagent

```
---
name: api-developer
description: Implement API endpoints following team conventions
skills:
  - api-conventions
  - error-handling-patterns
---
Implement API endpoints. Follow the conventions from the preloaded skills.
```

- The **full skill content** is injected at startup. Without `skills:` a subagent does **not** inherit skills you invoked, though it can still discover project/user/plugin skills through the Skill tool.
- Skills with `disable-model-invocation: true` cannot be preloaded. To stop a subagent using skills at all, omit `Skill` from `tools` or add it to `disallowedTools`.
- Inverse pattern: a skill with `context: fork` runs in a subagent. Either way the subagent starts without your history.

IN PRACTICE · APPLYING THIS TO TC-SUBAGENT

The script's `tc-subagent` says “apply the rules from the `tc-generator` skill” but has no `skills:` line. Adding `skills: [tc-generator]` guarantees the rulebook is in its context at startup instead of relying on it to go and find the skill — and it fits the “do not call tools” constraint because no Read is needed. Worth testing.

Scope MCP servers to one agent

```
mcpServers:
  - playwright: # inline: connects when the agent starts, gone when it ends
    type: stdio
    command: npx
    args: ["-y", "@playwright/mcp@latest"]
  - github # by name: reuses the session's existing connection
```

Defining a server **inline** keeps its tool descriptions out of the main conversation entirely — the subagent gets the tools, the parent does not. Project-level agent files load inline servers only after you **trust the folder**. Managed MCP policies and `--strict-mcp-config` still apply.

Persistent memory

MEMORY: SCOPE	LOCATION	USE WHEN
user	~/.claude/agent-memory/<agent>/	Learnings should follow the agent across all projects
project	.claude/agent-memory/<agent>/	Project-specific and shareable via version control (recommended default)
local	.claude/agent-memory-local/<agent>/	Project-specific but not committed

The agent's prompt gets memory instructions plus the first 200 lines / 25KB of its `MEMORY.md`, and Read/Write/Edit are enabled so it can maintain the files. It has no effect if auto memory is switched off.

Conditional rules with hooks

When `tools` is too blunt (allow some Bash, block the rest), use a `PreToolUse` hook. Claude Code passes the tool input as JSON on stdin; **exit code 2 blocks** the call and feeds stderr back to the agent.

frontmatter

```
---
name: db-reader
description: Execute read-only database queries.
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

scripts/validate-readonly-query.sh (chmod +x on macOS/Linux)

```
#!/bin/bash
INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')
if echo "$COMMAND" | grep -iE '\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRUNCATE)\b' >/dev/null; then
  echo "Blocked: only SELECT queries are allowed" >&2
  exit 2
fi
exit 0
```

- `SubagentStart` / `SubagentStop` hooks in `settings.json` fire when any (or a matched) agent starts or finishes; a `frontmatter Stop` is converted to `SubagentStop`.
- Project-level frontmatter hooks run only after you accept the workspace trust dialog for that folder.

PART 7 · COST & CAPABILITY

Choosing a Model

Each subagent can run on a different model — and the default may be pricier than you expect

Set `model:` to an alias (`sonnet`, `opus`, `haiku`, `fable`), a full model ID, or `inherit`. Claude Code resolves the model in this order:

#	SOURCE	NOTES
1	The per-invocation <code>model</code> parameter	Claude can pass one on a specific Agent call; it also sticks when the agent is resumed.
2	The agent's <code>model</code> frontmatter	<code>inherit</code> means “use the main conversation's model”.
3	<code>CLAUDE_CODE_SUBAGENT_MODEL</code> env var	A default only. Add <code>CLAUDE_CODE_SUBAGENT_MODEL_FORCE=1</code> (v2.1.257+) to force one model on every subagent.
4	The main conversation's model	The fall-through default.

- A family alias such as `opus` resolves to the **main conversation's exact model** when that model is in the same family (including a `[1m]` suffix).
- Organisation `availableModels` allowlists are checked; a blocked value is substituted and a warning is shown.
- Subagents **inherit extended-thinking** settings from the session (v2.1.198+); there is no per-agent thinking switch.
- Check what actually ran with `/tasks` while a subagent is running.

WARNING · COST TRAP

Because step 4 is “the main model”, a cheap triage or lint worker **silently runs on Opus if your session does**. Set `model:` `haiku` or `sonnet` explicitly on routine workers, and keep the expensive model for the coordinator.

Invoking & Delegating

Automatic delegation, explicit requests, and running a whole session as one agent

Automatic delegation runs on the description

Claude decides whether to delegate from three things: your request, each agent's `description`, and current context. The name and description of every available agent ride along in the main context on every turn, so the description is the **actual lever**:

- Want Claude to reach for it proactively? Put phrases like **“use proactively”** in the description.
- Want it to fire **only** from your pipeline? Keep it narrow — e.g. “spawned once per story only by the generate-test-cases plan; do not use standalone”.
- Descriptions cost context. When the combined descriptions of your **non-built-in** agents pass **15,000 tokens**, Claude Code warns at startup. Keep descriptions short; move detail into the body, which loads only when the agent runs.

Four ways to invoke an agent explicitly

WAY	EXAMPLE	GUARANTEE
Natural language	Use the <code>test-runner</code> subagent to fix failing tests	Claude usually delegates — its call.
@-mention	@“code-reviewer (agent)” look at the auth changes	That agent runs for this task. Claude still writes its task prompt from your message.
--agent flag	<code>claude --agent code-reviewer</code>	The whole session takes on that agent's prompt, tools and model. Its prompt replaces the default Claude Code prompt; <code>CLAUDE.md</code> still loads.
agent setting	<code>{ "agent": "code-reviewer" }</code> in <code>.claude/settings.json</code>	Same as <code>--agent</code> , as the project default; the CLI flag wins if both are set.

Plugin agents appear as `my-plugin:code-reviewer`; type `@agent-<name>` manually if you prefer. Resuming a session restores the chosen agent's tool limits and model.

Anatomy of an Agent tool call

```
Agent({ subagent_type: "tc-subagent",           // which agent to spawn
  description: "Generate TCs for story 1 (login)", // short label
  prompt:      '{ "story": "<the login story text>" }' }) // the payload it works from
// optional: name, model, isolation: "worktree", run_in_background
```

Omitting `subagent_type` fails with `subagent_type is required` when the session has no `general-purpose` agent to fall back on. Three calls in **one turn** = three subagents at once.

KEY IDEA · EXPLICIT BEATS GUESSED

In the coordinator pattern every spawn is **explicit, by name, from a numbered step**. The description stays narrow so Claude never auto-delegates “when do we generate test cases” on its own judgement — that is what keeps the pipeline deterministic.

Running Subagents

Foreground vs background, parallel patterns, limits, nesting, forks and resuming

Foreground vs background

	FOREGROUND	BACKGROUND
Main conversation	Blocks until it finishes.	Keeps running while you work; result arrives as a completion notification.
Permission prompts	Passed through to you as they occur.	Surfaced in your main session, naming the asking agent. Esc denies that one call.
Tools	Full inherited set.	Smaller built-in set (see Part 6).
Trigger	Claude needs the result before continuing.	Default. In interactive sessions fork mode is on and Claude's subagents run in the background. Ctrl+B backgrounds a running task; frontmatter background: true forces it.

Three patterns that work well

PATTERN	PROMPT SHAPE	BEST WHEN
Isolate high-volume output	Use a subagent to run the test suite and report only the failing tests with their errors	Logs, test output, fetched docs would swamp the main window.
Parallel research	Research the auth, database and API modules in parallel using separate subagents	The paths are independent.
Chain	Use the code-reviewer subagent to find performance issues, then the optimizer subagent to fix them	Each step needs the previous result — sequential by nature.

WARNING · RESULTS STILL LAND IN YOUR CONTEXT
Every subagent's summary returns to the main conversation, and each one spends tokens of its own. Many workers that each return a detailed report can consume real context. Return **compact, structured** output (the script's JSON-array contract) and prefer fewer, better-scoped workers.

Limits and isolation

SETTING	DEFAULT	NOTES
Concurrent subagents	20	Spawn #21 fails with <code>Concurrent subagent limit reached</code> . Change with <code>CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS</code> (v2.1.217+). No cap on total spawned over a session.
Nesting depth	3 layers below main	Set <code>CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH</code> ; 1 turns nesting off. At the limit the <code>Agent</code> tool is withheld, so the worker just does the job itself.
<code>isolation: worktree</code>	off	Gives the agent a temporary git worktree so parallel editors cannot collide; cleaned up if unchanged.

Forks: a subagent that keeps your context

A **fork** inherits the whole conversation instead of starting blank — same system prompt, tools, model and history, and it shares the parent's prompt cache, so it is cheaper than a cold subagent for context-heavy side tasks. Start one with `/subtask <task>` (v2.1.212+; earlier versions used `/fork`). Only its final result comes back. A fork cannot spawn further forks.

	FORK	NON-FORK SUBAGENT
Context	Full conversation history	Fresh, plus the prompt you pass
Prompt & tools	Same as main session	From its definition file
Model	Same as main session	From its <code>model</code> field
Prompt cache	Shared with main	Separate

Resuming, transcripts and compaction

- Each invocation is a **new instance**. To continue one, ask Claude to resume it: it sends a message with `SendMessage` using the agent ID or name, and the agent picks up with its full previous history.
- Transcripts live at `~/.claude/projects/{project}/{sessionId}/subagents/agent-{agentId}.jsonl` and survive main-conversation compaction; they are deleted after `cleanupPeriodDays` (30 by default).
- Subagents auto-compact by the same rules as the main session. A worker stopped by `maxTurns` returns **partial** output that Claude can resume.
- If an API error cuts a worker off, Claude gets a failure (background) or the partial text with a cut-off note (foreground) — not error text disguised as findings.
- From v2.1.210, Claude Code scans each subagent's final report for text imitating system tags and marks it as subagent output, since workers may have read untrusted content.

PART 10 · WORKED EXAMPLE A

Coordinator + tc-subagent

Turning the test-case generator into a split → fan-out → merge → write pipeline

Goal: paste 3 user stories; get one file of test cases. Before: one Claude, one window, stories done one after another. After: `CLAUDE.md` becomes a **coordinator** that never generates test cases itself; a `tc-subagent` per story does the work in parallel.

The nine-step flow

#	STEP	WHAT HAPPENS
1	Entry	<code>/generate-test-cases</code> + 3 stories matches <code>.claude/commands/generate-test-cases.md</code> ; its body (with <code>\$ARGUMENTS</code> substituted) becomes the turn's instruction: follow the coordinator plan in <code>CLAUDE.md</code> .
2	Coordinator loads	Main Claude reads <code>CLAUDE.md</code> : split, fan out, merge, write — and “I do NOT generate test cases myself”.
3	Split	Parse <code>\$ARGUMENTS</code> into story chunks (blank line / <code>Story:</code> label / change of subject). No generation yet.
4	Fan out	One <code>Agent</code> call per chunk with <code>subagent_type</code> : “tc-subagent” and <code>{ "story": "<chunk>" }</code> , all in the same turn → 3 subagents start together.
5	Definition loads	Claude Code reads <code>.claude/agents/tc-subagent.md</code> for each; the payload is its first input. None can see the others or the coordinator's context.
6	Inside each worker	Applies the <code>tc-generator</code> rules, reasons about its one story, drafts cases to the coverage minimums, self-checks, returns a JSON array only.
7	Hand-back	Arrays arrive asynchronously, whenever each finishes (so completion order ≠ story order).
8	Merge	Flatten, keep story order, drop exact-duplicate titles, assign IDs <code>TC-{storyN}-{TYPE}-{seq}</code> .
9	Write	Render each object into the skill's markdown block and write <code>output/tc_YYYY-MM-DD.md</code> , then print a summary (stories in, cases out, by type, path).

The coordinator plan (CLAUDE.md, condensed)

```
# Test Case Generator - Coordinator
I split the stories, hand each to a subagent, merge the results, and write one file.
I do NOT generate test cases myself.

### 1. Split - blank line / "Story:" label / change of subject = boundary; keep verbatim
### 2. Fan out - for EACH story spawn `tc-subagent` via the Agent tool, by name, in parallel
### 3. Merge - flatten, keep order, drop duplicate titles, id = TC-{storyN}-{TYPE}-{seq}
### 4. Write - output/tc_YYYY-MM-DD.md in the tc-generator skill's format; print a summary

## If a subagent fails
Log `[SKIP] story N: <reason>` and carry on. Never abort the whole run for one.
```

Design decisions worth explaining

DECISION	WHY
"I do NOT generate test cases myself"	Load-bearing. Without it the main Claude quietly does story 1 itself and forgets to fan out.
Return JSON array only, no prose or fences	The coordinator is code that must parse and merge results. The contract between agents is data, never chat .
JSON fields mirror the skill's markdown fields 1:1	The coordinator transplants fields into a template by name — it never needs to read <code>SKILL.md</code> , so that work is done once, inside each worker.
Narrow, "do not use standalone" description	Prevents auto-delegation outside the pipeline.
Failure rule in the plan ([SKIP])	One bad chunk (e.g. <code>asdfgh</code>) is logged and skipped; the other stories still ship.
Workers do not write files	Workers produce data; the coordinator owns the output file.

IN PRACTICE · THE PAYOFF, IN NUMBERS

Time: run sequentially, 3 stories take about 3× one story; in parallel, about 1× (a gap that widens at 10 or 20 stories). Cold-start cost per worker means the real ratio is approximate — time it on your own repo.

Context (the script's illustrative estimate): each worker may peak around 15–25k tokens (skill + reasoning + drafts) but hand back only a 2–4k-token JSON array. Inline, the main window would hold ~45–75k tokens of scratch work for 3 stories; with subagents it holds ~6–12k. `/context` only measures your session, so it cannot see inside a closed subagent — which is why the bars look so different.

PART 11 · WORKED EXAMPLE B

spec-reviewer + security-agent

Two different worker types reviewing one pull request in a Playwright framework

Situation: `/review` checks changed files against the `automation-standards` skill, but in one context — file 6 is reviewed in a window already full of files 1–5. The team also wants credential, secret and auth-practice checks. The tempting move is to append a security checklist to the skill. **Don't:** a new kind of review gets **its own subagent**, not a new paragraph in an existing one.

	SPEC-REVIEWER	SECURITY-AGENT
Scope	One spec or page object per worker	The whole changeset in a single worker
tools	Read	Read, Grep (what a pattern sweep needs)
Checks	Every standard in <code>automation-standards</code> (waits, selectors, isolation, Page Objects, data...)	Hardcoded secrets/keys/tokens, PII in test data, auth-testing practices, insecure config (TLS off, <code>http://</code> , over-permissive roles, no cleanup)
Returns	JSON: <code>path</code> , <code>line</code> , <code>standard</code> , <code>why</code> , <code>fix</code> , <code>severity</code>	JSON: <code>path</code> , <code>line</code> , <code>kind</code> , <code>evidence (masked)</code> , <code>severity</code> , <code>fix</code>
Guardrails	Read only; one file; <code>[]</code> if clean	Read only; mask every secret except last 4 chars ; <code>[]</code> if clean

The fan-out in review.md

```
## Step 1 - Identify target files (paths from $ARGUMENTS, else files changed vs main)
## Step 2 - Fan out, all in parallel
- one `spec-reviewer` subagent per changed spec / page object, one path each
- one `security-agent` subagent over the whole list of changed files
## Step 3 - Merge results
Collect every JSON array. If both agents flag the same line, keep it once at the higher severity.
## Step 4 - Report
Findings grouped by file, a short "Security" section, then a one-line verdict:
BLOCK / FIX-BEFORE-MERGE / NITS-ONLY / PASS
## Step 5 - Failure handling
Log `[SKIP] <name/path>` and keep the rest. Do not modify any files.
```

- For a 3-file diff, **four workers launch at once — two kinds** (3 × spec-reviewer + 1 × security-agent).
- The merge step does real work: the hardcoded `secret_sauce` password is flagged by security-agent **and** by spec-reviewer under a “data not literals” standard — one merged finding at security severity.
- Run `/review` with no arguments from a pre-push hook or CI and you have a standards-and-security gate that scales to a real PR.

INFO · WHERE THE PLAN LIVES IS A CONVENTION, NOT A RULE

CLAUDE.md always loads at session start; a slash command loads only when typed. `tc-generator` puts the plan in CLAUDE.md (its command file is one line); `pw-automation-lab` puts it in `review.md` and leaves a one-line pointer in CLAUDE.md. Both work — pick one convention per project and don’t split a single command’s plan across both files.

PART 12 · REFERENCE

Subagents vs Skills vs Commands vs CLAUDE.md

Four mechanisms, four different jobs — and they compose

	CLAUDE.MD	SLASH COMMAND	SKILL	SUBAGENT
Loads	Always, at session start	Only when typed	Description always; body when triggered	Description always; body only when it runs
Runs in	Main context	Main context	Main context	Its own context window
Triggered by	N/A (always on)	You	You or Claude (from the description)	Claude, you (@-mention, <code>--agent</code>), or a coordinator plan
Tool control	None	None	<code>allowed-tools</code> pre-approves only	<code>tools</code> / <code>disallowedTools</code> restrict
Best for	Standing facts true on almost every task	A short, predictable front door	Rulebooks / processes needed sometimes	Noisy, self-contained work; parallel jobs; hard tool limits
Example	Repo overview, commands	<code>/generate-test-cases</code>	<code>tc-generator</code>	<code>tc-subagent</code>

KEY IDEA · THEY COMPOSE, NOT COMPETE

CLAUDE.md says what the repo is and holds the coordinator plan → the **skill** holds the rulebook → the **subagent** does one job in a fresh context and returns data → the **command** gives the whole chain a short name. Chain: `/generate-test-cases` → coordinator plan → N × `tc-subagent` (parallel) → each applies `tc-generator` → merge → file.

Subagent or main conversation?

USE THE MAIN CONVERSATION WHEN...	USE A SUBAGENT WHEN...
The task needs frequent back-and-forth or iterative refinement.	The task produces verbose output you do not need in your main context.
Several phases share a lot of context (plan → implement → test).	You want to enforce specific tool restrictions or permissions.
It is a quick, targeted change.	The work is self-contained and can return a summary.
Latency matters — a cold subagent needs time to gather context.	You can run several independent jobs at once.

For a question about something **already in the conversation**, use `/btw` instead: it sees your full context, has no tools, and the answer is not added to history. For reusable prompts that should run in the main context, use a **skill**.

Beyond a single session

FEATURE	USE IT FOR
Subagents	Delegated workers inside one session ; only summaries return.
Background agents (agent view)	Many independent sessions running in parallel, monitored from one place.
Cross-session messaging	Separate sessions that pass messages to each other.
Agent teams	A coordinated team of sessions that Claude spawns and supervises.
Worktrees	Isolated checkouts so parallel edits do not collide.

PART 13 · WATCH-OUTS

Gotchas & Refinements

Where the Episode script can be tightened against the official docs

Script says → the docs add

THE SCRIPT SAYS	REFINEMENT FROM THE OFFICIAL DOCS
"A subagent starts blank — nothing from your chat history."	True for history . CLAUDE.md, git status (except Explore/Plan), the agent's prompt and any preloaded skills still load. Use <code>omitClaudeMd: true</code> to drop CLAUDE.md.
"Subagents can't talk to each other, or to you, mid-run."	The design pattern is still inputs in / output out. Mechanically: foreground permission prompts reach you; agents with <code>SendMessage</code> can message named agents; nested subagents (3 layers by default) report to their launcher.
"Omit <code>tools</code>: and it gets the whole pool."	Correct. Also: no frontmatter gives zero tools; an unresolvable list usually fails to launch (v2.1.208+); background runs get a reduced built-in set regardless.
"Every message includes every agent's name and description."	Correct — and they cost context. Over 15,000 tokens of combined non-built-in descriptions triggers a startup warning.
"<code>tools: Read</code> physically prevents edits."	Correct for subagents (allowlist). Contrast with a skill's <code>allowed-tools</code> , which only pre-approves.
"Three calls in the same turn run at once."	Yes — in interactive sessions they run in the background by default, up to 20 concurrent. Each pays its own start-up cost (CLAUDE.md, skills).
"The agent applies the <code>tc-generator</code> skill."	Make it deterministic with <code>skills: [tc-generator]</code> (full content preloaded) rather than hoping it finds the skill.

Classic ways this goes wrong

- **Coordinator does the work itself.** Keep the explicit “I do NOT do the work myself” line; otherwise story 1 gets done inline and the fan-out is skipped.
- **Loose description.** A vague `description` invites auto-delegation somewhere you never intended. Scope it to one pipeline.
- **Parallelising dependent jobs.** If worker two needs worker one’s result, run them in sequence; forcing parallel just breaks it.
- **Token bill.** Each worker fills its own window and reloads `CLAUDE.md` and skills. You trade tokens for wall-clock time and isolation — usually worth it, not always.
- **Underspecified prompts.** A cold worker with a thin prompt wastes its first turns rediscovering context; put everything it needs in the payload.
- **Prose is not enforcement.** “Do not write files” is a request; `tools` and hooks are enforcement. Use frontmatter for anything that must hold.
- **Shared checkout.** Parallel agents that write files can collide — use `isolation: worktree` or give each an owned directory.

Glossary

TERM	MEANING
Subagent	A separate Claude instance with its own context window, system prompt, tool access and permissions; returns a summary.
Coordinator	The main Claude when it splits work, fans out subagents, merges and writes — without doing the work itself.
Fan-out	Spawning several subagents in the same turn so they run concurrently.
Agent tool	The tool that spawns subagents (formerly Task, renamed in v2.1.63).
<code>subagent_type</code>	The Agent-call parameter naming which agent to spawn.
Frontmatter	The YAML block between <code>---</code> markers that configures an agent.
Fork	A subagent that inherits the full conversation and shares the parent's prompt cache (<code>/subtask</code>).
Foreground / background	Whether a subagent blocks the main conversation or runs alongside it.
<code>SendMessage</code>	The tool used to resume or message an existing subagent by ID or name.
Worktree isolation	<code>isolation: worktree</code> — a temporary git worktree so an agent edits an isolated copy of the repo.
Progressive disclosure	Always-visible short metadata, detailed body loaded only when needed — same idea as skills.

Self-test

#	QUESTION
1	Why does reviewing 50 files in the main conversation hurt later, unrelated requests — even with a very large context window?
2	Name four things a non-fork subagent starts with, and three things it does not get.
3	You omit <code>tools:</code> from an agent file. What can it use? Can you force zero tools from frontmatter?
4	How does a subagent's <code>tools: Read</code> differ from a skill's <code>allowed-tools: Read</code> ?
5	What are the four sources in the subagent model order, and why might a triage agent run on Opus without you noticing?
6	Give the three parts of an Agent tool call, and explain what “in the same turn” buys you.
7	Why does the coordinator plan say “I do NOT generate test cases myself”, and why must workers return a JSON array only?
8	In <code>/review</code> , why is <code>security-agent</code> a separate worker instead of extra rules in <code>automation-standards</code> ?

Sources: Anthropic, “Create custom subagents” (Claude Code docs, code.claude.com/docs/en/sub-agents); Anthropic Academy, “Introduction to subagents”; Episode script and project agent files (tc-subagent, spec-reviewer, security-agent). Version numbers are from the docs at the time of writing.